Heart Disease — Preprocessing & Feature Engineering

Dataset: UCI Heart Disease (Cleveland processed)

Sections:

1. Load cleaned dataset
2. Build the preprocessing pipeline
3. Fit & transform — inspect numeric vs categorical handling
4. Missing-value handling demo
5. Full Pipeline (preprocessor + classifier)

In [1]:
```
import sys
from pathlib import Path
import numpy as np
import pandas as pd

ROOT = Path.cwd().parent if Path.cwd().name == 'notebooks' else Path.cwd()
sys.path.insert(0, str(ROOT))

from src.data_loader import (
    load_clean, split_features_target,
    NUMERIC_FEATURES, CATEGORICAL_FEATURES, TARGET,
)
from src.preprocess import build_preprocessor, build_pipeline

print('Numeric features    :', NUMERIC_FEATURES)
print('Categorical features:', CATEGORICAL_FEATURES)
```

```
Numeric features    : ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']
Categorical features: ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'tha
l']
```

1. Load cleaned dataset

In [2]:
```
df = load_clean()
X, y = split_features_target(df)
print('X shape:', X.shape, '| y shape:', y.shape)
print('Target distribution:')
print(y.value_counts())
X.head()
```

```
X shape: (303, 13) | y shape: (303,)
Target distribution:
target
0    164
1    139
Name: count, dtype: int64
```

Out[2]:

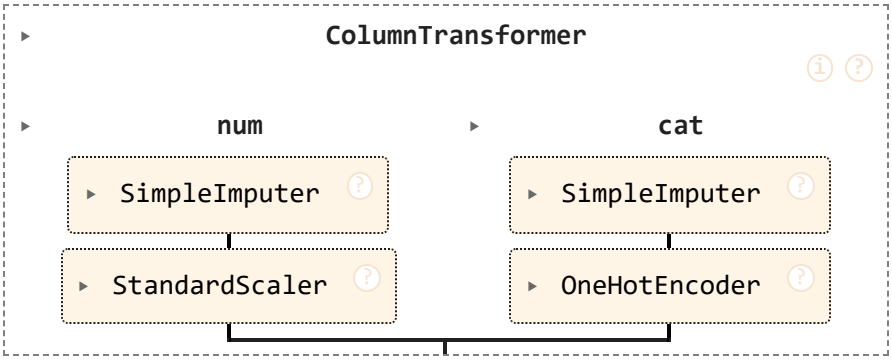
	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	63.0	1.0	1.0	145.0	233.0	1.0	2.0	150.0	0.0	2.3	3.0	0.0	6.0
1	67.0	1.0	4.0	160.0	286.0	0.0	2.0	108.0	1.0	1.5	2.0	3.0	3.0
2	67.0	1.0	4.0	120.0	229.0	0.0	2.0	129.0	1.0	2.6	2.0	2.0	7.0
3	37.0	1.0	3.0	130.0	250.0	0.0	0.0	187.0	0.0	3.5	3.0	0.0	3.0
4	41.0	0.0	2.0	130.0	204.0	0.0	2.0	172.0	0.0	1.4	1.0	0.0	3.0

2. Build the preprocessing pipeline

Numeric columns → `SimpleImputer(median)` → `StandardScaler`.
Categorical columns → `SimpleImputer(most_frequent)` → `OneHotEncoder(handle_unknown='ignore')`.

In [3]:
```
pre = build_preprocessor()
pre
```

Out[3]:



3. Fit and transform — inspect both branches

In [4]:
```
X_t = pre.fit_transform(X)
feat_names = pre.get_feature_names_out()
print('Transformed shape :', X_t.shape)
print('Total features after one-hot:', len(feat_names))
print('\nNumeric block (first 5 rows, scaled to mean 0 / std 1):')
pd.DataFrame(X_t[:5, :len(NUMERIC_FEATURES)],
             columns=feat_names[:len(NUMERIC_FEATURES)]).round(3)
```

```
Transformed shape : (303, 28)
Total features after one-hot: 28

Numeric block (first 5 rows, scaled to mean 0 / std 1):
```

Out[4]:

	age	trestbps	chol	thalach	oldpeak
0	0.949	0.758	-0.265	0.017	1.087
1	1.392	1.611	0.760	-1.822	0.397
2	1.392	-0.665	-0.342	-0.902	1.346
3	-1.933	-0.096	0.064	1.637	2.123
4	-1.489	-0.096	-0.826	0.981	0.311

In [5]:
```
print('Categorical block (one-hot, first 5 rows):')
pd.DataFrame(X_t[:5, len(NUMERIC_FEATURES):],
             columns=feat_names[len(NUMERIC_FEATURES):]).astype(int)
```

```
Categorical block (one-hot, first 5 rows):
```

Out[5]:

	sex_0.0	sex_1.0	cp_1.0	cp_2.0	cp_3.0	cp_4.0	fbs_0.0	fbs_1.0	restecg_0.0	restecg_1.
0	0	1	1	0	0	0	0	1	0	
1	0	1	0	0	0	1	1	0	0	
2	0	1	0	0	0	1	1	0	0	
3	0	1	0	0	1	0	1	0	1	
4	1	0	0	1	0	0	1	0	0	

5 rows × 23 columns

4. Missing-value handling demo

Inject `NaN` into one numeric and one categorical cell, then verify the pipeline imputes them.

In [6]:
```
sample = X.iloc[:3].copy()
sample.loc[sample.index[0], 'chol'] = np.nan  # numeric NaN
sample.loc[sample.index[1], 'thal'] = np.nan  # categorical NaN
print('NaNs in input :', int(sample.isna().sum().sum()))
out = pre.transform(sample)
print('NaNs in output:', int(np.isnan(out).sum()))
print('Imputation: numeric -> median, categorical -> mode.')
```

```
NaNs in input : 2
NaNs in output: 0
Imputation: numeric -> median, categorical -> mode.
```

5. Full Pipeline (preprocessor + classifier)

`build_pipeline(estimator)` glues the preprocessor onto the front of any sklearn estimator so a single `joblib.dump` ships every preprocessing step alongside the trained

model.

In [7]:
```
from sklearn.linear_model import LogisticRegression
pipe = build_pipeline(LogisticRegression(max_iter=1000))
pipe
```

Out[7]: